

Correction — Exercice 10 : Cascade et conflits

Solution — Le lien `.link-cta` est vert

Calcul de spécificité pour les 3 règles qui ciblent `<a class="link-cta">` :

Sélecteur	Spécificité (id, classe, type)	Total numérique
<code>a</code>	0,0,1	1
<code>.link-cta</code>	0,1,0	10
<code>#content a</code>	1,0,1	101

→ `#content a` **gagne** avec 101. Le lien apparaît en **vert**.

Le lien standard `<a href="#">` n'est pas ciblé par `.link-cta`, donc seulement par `a` (1) et `#content a` (101). Il est aussi vert.

Leçon : un id pèse l'équivalent de 100 classes. Ajouter un id dans un sélecteur a un effet « bombe » sur la spécificité — à éviter en CSS applicatif.

Solution — Partie A : Calculs de spécificité

Sélecteur	Spécificité (ID, classe, type)	Score final
<code>p</code>	0,0,1	1
<code>.card</code>	0,1,0	10
<code>#footer</code>	1,0,0	100
<code>ul li.active</code>	0,1,2	12
<code>nav > ul > li a:hover</code>	0,1,4	14
<code>#app .sidebar .menu a</code>	1,2,1	121
<code>input[type="email"]:focus</code>	0,2,1	21
<code>body.logged-in .header</code>	0,2,1	21

Rappel méthode :

- Compter les IDs (`#...`) → 1ère colonne (× 100).
- Compter les classes (`.xxx`), attributs (`[xxx]`), pseudo-classes (`:xxx`) → 2ème colonne (× 10).
- Compter les types (`div` , `p`) et pseudo-éléments (`::before`) → 3ème colonne (× 1).
- Combinateurs (`>` , `+` , `~` , `,`) : **aucun impact** sur la spécificité.
- `*` (sélecteur universel) : spécificité 0.

Solution — Partie B : bug-fixed.css

Diagnostic

Règles ciblant `<h2 class="titre">` dans `<article class="article" id="hero">` :

Sélecteur	Spécificité	Couleur	Statut
<code>body</code> (hérité)	0,0,1	black	écrasé par tous les autres
<code>.article</code> (hérité)	0,1,0	gray	écrasé
<code>article p</code>	0,0,2	darkblue	ne cible pas h2
<code>.article .titre !important</code>	0,2,0	red	gagne grâce au <code>!important</code>
<code>#hero .titre</code>	1,1,0	purple	écrasé par <code>!important</code>
<code>[data-theme="dark"] .titre</code>	0,2,0	white	écrasé par <code>!important</code>

→ Le **titre est rouge**.

Pour le paragraphe : seules `body` (héritage) et `article p` ciblent. → **darkblue**.

Refactor (sans `!important`)

```
/* bug-fixed.css
*
* Objectif : titre en vert (#0a8a4a), paragraphe inchangé (darkblue).
* Choix : spécificité minimale, aucune dépendance à id, aucun !important.
* On supprime le mode dark spécifique (hors-scope) et la règle .article.titre rouge.
*/

body { color: black; }

.article { color: gray; }

article p { color: darkblue; }

.titre { color: #0a8a4a; }
```

Points clés :

- Toutes les règles tiennent en **spécificité classe** maximum (0,1,0 ou 0,2,0). Pas d'id, pas d'attribut, pas d' **!important** .
- L'ordre des règles **n'importe pas** ici (chaque règle a un sélecteur distinct).
- Le mode dark `[data-theme="dark"]` a été retiré : s'il était nécessaire, on le mettrait dans un fichier `theme-dark.css` chargé conditionnellement, ou on utiliserait des **variables CSS** pour le titre :

```
:root { --titre-couleur: #0a8a4a; }  
[data-theme="dark"] { --titre-couleur: #4af28a; }  
.titre { color: var(--titre-couleur); }
```

→ cette approche **élimine** le besoin de cascade par sélecteur attribut.

Solution — @layer sur DigiShop

Refactor avec layers

```
/* digiblog/style-layers.css */

@layer reset, base, composants, utilities;

@layer reset {
  *, *::before, *::after { box-sizing: border-box; }
  body { margin: 0; }
}

@layer base {
  :root {
    --bleu: #2563eb;
    --orange: #ea580c;
    --gris: #6b7280;
  }
  body {
    font-family: system-ui, sans-serif;
    color: #111;
    line-height: 1.6;
  }
  h1, h2 { font-family: 'Playfair Display', serif; }
  /* Un id agressif ici n'écrasera pas une simple classe des composants ! */
  #main-content h2 { color: red; }
}

@layer composants {
  .article {
    border: 1px solid #e5e7eb;
    padding: 1.5rem;
  }
  .article-titre {
    color: var(--orange);
    /* Cette règle gagne sur #main-content h2 du layer base */
  }
  .btn-cta {
    color: var(--orange);
    text-decoration: underline;
  }
}

@layer utilities {
  .text-center { text-align: center; }
  .mb-2 { margin-bottom: 0.5rem; }
}
```

Synthèse pédagogique

Règles d'or de la cascade :

1. **Spécificité minimale viable** : la règle qui gagne doit être la plus simple possible.
2. **Pas d'id en CSS applicatif** (réserver aux ancres anchor `#section1`).
3. **!important jamais** sauf utilities (et encore, `@layer` rend même ça superflu).
4. **Conventions de nommage > sélecteurs imbriqués** (BEM, classes uniques).
5. **Cascade Layers** pour les codebases > 500 lignes — ça change la donne.